PRACTICAL GUIDE

Building Android Screens

A practical Jetpack Compose guide for working in an Android team

Jetpack Compose | ViewModel | Navigation | Prepared for **Anjana**

This guide teaches you how to build real Android screens the way professional teams do: with **Jetpack Compose**, **ViewModels**, **UI state** and **Navigation**. Every chapter starts from the basics and ends with code you would actually write at work.

Chapter	Topic	At work you'll use it to...
1	How screens work in Compose	Understand any screen file you open
2	Basic components	Show text, icons, buttons, cards
3	Layouts: Column, Row, Box	Arrange anything a designer draws
4	Modifiers	Control size, spacing, shape, clicks
5	Screen structure with Scaffold	Add top bars, bottom bars, FABs
6	Lists	Show feeds, contacts, products
7	State & user input	Build forms such as login and sign-up
8	ViewModel & UI state (team pattern)	Connect screens to real data correctly
9	Navigation	Move between screens and pass data
10	Theme, colors & strings	Follow the design system, support dark mode
11	Real-world screen recipes	Settings, search, dialogs, snackbars, loading
12	Working in a team	Find code, follow conventions, pass code review
13	Reading XML layouts	Work on older parts of the app

How to use this guide

- Create a practice project in Android Studio (**New Project** → **Empty Activity**) and type every example into it.
- Look at each example in the **Preview** panel (Split view), then run it on a phone or emulator.
- Do the **practice tasks** at the end of each chapter. They are similar to real tickets.
- Compare each chapter with your team's real code. You'll start recognising the same patterns.

> **Tip:** Every Compose example in this guide was **compile-checked** with the Compose compiler. **Import lines are left out** to keep the code short: when a word is red in Android Studio, click it and press **Alt + Enter** → **Import**. Appendix A lists the common imports.

> **Careful:** Your team's project is the final authority. If your team does something differently from this guide (for example uses Hilt, or a custom design system), follow the team. This guide shows the standard, most common way.

Box colour	Means
Purple bar	Compose code, compile-checked: type it into your practice project
Blue background	Android-only code (Activity, R.string, Hilt, Views): shown for reference
Yellow bar	XML files (layouts, strings)
White box, grey border	A sketch of what the screen looks like
Red background	Wrong code: a bug or a bad practice to avoid

CHAPTER 1

How Screens Work in Compose

Goal: Understand composable functions, the UI tree, previews and where screen code lives in a team project.

1. A screen is made of functions

In Jetpack Compose, you build the UI by writing **Kotlin functions** marked with **@Composable**. Each function **describes** a piece of the screen. You combine small functions into bigger ones, like LEGO blocks.

The smallest composable

```
@Composable
fun Greeting(name: String) {
    Text(text = "Hello, $name!")
}

@Preview(showBackground = true)
@Composable
fun GreetingPreview() {
    Greeting(name = "Anjana")
}
```

Part	Meaning
@Composable	This function draws UI. It can only be called from another @Composable function.
Function name	Starts with a **capital letter** and is a noun: Greeting, ProfileCard, LoginScreen.
Parameters	The **data** the UI needs (name). The UI shows whatever you pass in.
Text(...)	A built-in composable from Material 3.
@Preview	Shows the composable in Android Studio's Preview panel without running the app.

2. Combining composables

A card built from smaller pieces

```
@Composable
fun ProfileCard(name: String, role: String, city: String) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text(text = name, style = MaterialTheme.typography.titleLarge)
        Text(text = role, style = MaterialTheme.typography.bodyMedium)
        Text(text = city, style = MaterialTheme.typography.bodySmall)
    }
}

@Preview(showBackground = true)
@Composable
fun ProfileCardPreview() {
    ProfileCard(name = "Anjana", role = "Android Developer", city = "Kochi")
}
```

Sketch: What it looks like

```
+------------------------------+
|  Anjana                      |   <- titleLarge
|  Android Developer           |   <- bodyMedium
|  Kochi                       |   <- bodySmall
+------------------------------+
```

Compose builds a **tree** from your functions. Reading the code from top to bottom gives you the tree:

Sketch: The UI tree

```
ProfileCard
 +-- Column
      +-- Text("Anjana")
      +-- Text("Android Developer")
      +-- Text("Kochi")
```

3. The most important idea: UI = f(state)

A composable takes data and turns it into UI. When the data changes, Compose calls the function again and the screen updates automatically. This is called **recomposition**. You never write "change this label's text". You change the **data**, and the UI follows.

Sketch: Data flows into the UI

```
   data (state)                    composable                      screen
 name = "Anjana"   ------->   ProfileCard(name, ...)   ------->   shows "Anjana"
 name = "Rahul"    ------->   ProfileCard(name, ...)   ------->   shows "Rahul"   (automatic)
```

4. Where the screen starts: MainActivity

MainActivity.kt (Android only)

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            MyAppTheme {            // your team's theme
                AppNavHost()        // the navigation graph decides which screen to show
            }
        }
    }
}
```

In most team apps, **MainActivity is tiny**. It sets the theme and the navigation graph, and all the real work happens in screen composables. You will rarely change MainActivity.

5. Where screen code lives in a team project

Sketch: A typical feature-based structure

```
app/src/main/java/com/company/app/
 |-- MainActivity.kt
 |-- navigation/
 |    +-- AppNavHost.kt            <- all routes and screens connected here
 |-- ui/
 |    |-- theme/                   <- Color.kt, Theme.kt, Type.kt (design system)
 |    +-- components/              <- shared buttons, cards, top bars used by many screens
 |-- feature/
 |    |-- login/
 |    |    |-- LoginScreen.kt      <- the UI (composables)
 |    |    |-- LoginViewModel.kt   <- logic and state
 |    |    +-- LoginUiState.kt     <- the data the screen shows
 |    +-- home/ ...
 +-- data/                         <- repositories, API calls, database
```

Tip: When you get a new task, first open the **feature folder** for that screen and the **ui/components** folder. Most of what you need (buttons, cards, colors) already exists. Reuse it instead of creating a new one.

6. Using Preview effectively

- Click **Split** (top right of the editor) to see code and preview side by side.
- Previews update when you change code. Use **Build & Refresh** if a preview gets stuck.
- Write a preview for every screen **state** (loading, error, content). Chapter 8 shows how.
- Previews must not need real data, so pass **fake data** as parameters.

Practice Tasks

1. Create a composable **BookCard(title, author, year)** that shows three lines of text with different typography styles. Add a preview.

2. Open your team's project. Find MainActivity and follow it to the navigation graph. Write down the names of 3 screens.

3. In your team's project, find the **ui/theme** (or design system) folder and list the files in it.

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 2

Basic Components

Goal: Use Text, Icon, Image, buttons, Card, Divider and Spacer correctly.

1. Text

Text styles

```
@Composable
fun TextExamples() {
    Column(
        modifier = Modifier.padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Text("Headline", style = MaterialTheme.typography.headlineMedium)
        Text("Title", style = MaterialTheme.typography.titleMedium)
        Text("Body text that explains something.", style = MaterialTheme.typography.bodyLarge)
        Text(
            text = "LABEL",
            style = MaterialTheme.typography.labelSmall,
            color = MaterialTheme.colorScheme.primary
        )
        Text(
            text = "Bold and centred",
            fontWeight = FontWeight.Bold,
            textAlign = TextAlign.Center,
            modifier = Modifier.fillMaxWidth()
        )
        Text(
            text = "A very long text that does not fit on one line will end with three dots",
            maxLines = 1,
            overflow = TextOverflow.Ellipsis
        )
    }
}
```

Tip: In a team, **always use MaterialTheme.typography** styles instead of setting fontSize by hand. Then the whole app changes in one place when the designer updates the fonts.

Style	Typical use
displayLarge / Medium / Small	Very big numbers or hero text
headlineLarge / Medium / Small	Screen headings
titleLarge / Medium / Small	Card titles, top bar titles, list item titles
bodyLarge / Medium / Small	Normal paragraph text
labelLarge / Medium / Small	Buttons, chips, captions

2. Buttons

Button types

```
@Composable
fun ButtonExamples() {
    Column(
        modifier = Modifier.padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        Button(onClick = { }) { Text("Button (main action)") }
        FilledTonalButton(onClick = { }) { Text("Tonal button") }
        OutlinedButton(onClick = { }) { Text("Outlined button") }
        TextButton(onClick = { }) { Text("Text button") }
        Button(onClick = { }, enabled = false) { Text("Disabled") }
        Button(onClick = { }) {
            Icon(Icons.Default.Add, contentDescription = null)
            Spacer(Modifier.width(8.dp))
            Text("With icon")
        }
        Row {
            IconButton(onClick = { }) {
                Icon(Icons.Default.Favorite, contentDescription = "Like")
            }
            IconButton(onClick = { }) {
                Icon(Icons.Default.Share, contentDescription = "Share")
            }
        }
    }
}
```

Button	When to use (Material guidelines)
Button	The **one** main action on a screen: Save, Login, Pay
FilledTonalButton	Important but secondary action
OutlinedButton	Secondary action next to a main button: Cancel
TextButton	Low-priority action: Forgot password?, Skip, dialog buttons
IconButton	An icon-only action, e.g. in a top bar. **Always** give a contentDescription

3. Icons

Icons.Default.X (same as Icons.Filled.X) gives a filled icon. **Icons.Outlined.X** gives an outlined one. Common icons (Add, Search, Home, Person, Settings, Delete, Favorite...) are in **material-icons-core**. Others (Visibility, CameraAlt...) need the **material-icons-extended** dependency. Your team may also use its own icons from res/drawable.

Icon sizes and colours

```
@Composable
fun IconExamples() {
    Row(
        modifier = Modifier.padding(16.dp),
        horizontalArrangement = Arrangement.spacedBy(16.dp)
    ) {
        Icon(Icons.Default.Home, contentDescription = "Home")
        Icon(Icons.Outlined.Home, contentDescription = "Home")
        Icon(
            imageVector = Icons.Default.Star,
            contentDescription = "Rating",
            tint = MaterialTheme.colorScheme.primary,
            modifier = Modifier.size(32.dp)
        )
    }
}
```

Tip: contentDescription is read aloud by TalkBack (the screen reader). Use a short description for icons that do something, and **null** for decorative icons that sit next to text saying the same thing.

4. Images

Images from resources and from the internet (Android only)

```
// From res/drawable (e.g. res/drawable/banner.png)
Image(
    painter = painterResource(id = R.drawable.banner),
    contentDescription = "Summer sale banner",
    contentScale = ContentScale.Crop,          // fill the space, cut off the edges
    modifier = Modifier
        .fillMaxWidth()
        .height(180.dp)
        .clip(RoundedCornerShape(12.dp))
)

// From a URL: most teams use the Coil library
AsyncImage(
    model = user.photoUrl,
    contentDescription = "Profile photo",
    contentScale = ContentScale.Crop,
    modifier = Modifier.size(48.dp).clip(CircleShape)
)
```

ContentScale	Effect
Crop	Fills the whole area, cuts off the extra parts (most common for photos)
Fit	Shows the whole image, may leave empty space
FillBounds	Stretches the image (usually looks bad)

5. Card, Divider and Spacer

An order card

```
@Composable
fun OrderCard() {
    Card(modifier = Modifier.fillMaxWidth().padding(16.dp)) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text("Order #1024", style = MaterialTheme.typography.titleMedium)
            Spacer(Modifier.height(4.dp))
            Text("2 items - Rs 450", style = MaterialTheme.typography.bodyMedium)
            HorizontalDivider(modifier = Modifier.padding(vertical = 12.dp))
            Row(verticalAlignment = Alignment.CenterVertically) {
                Icon(
                    imageVector = Icons.Default.CheckCircle,
                    contentDescription = null,
                    tint = MaterialTheme.colorScheme.primary
                )
                Spacer(Modifier.width(8.dp))
                Text("Delivered")
            }
        }
    }
}
```

Sketch: What it looks like

```
+----------------------------------------+
|  Order #1024                           |
|  2 items - Rs 450                      |
|  ------------------------------------  |
|  (v) Delivered                         |
+----------------------------------------+
```

Material 3 also has **ElevatedCard** (with a shadow) and **OutlinedCard** (with a border). **Surface** is a plain background container. Cards are just Surfaces with a shape and colour.

Practice Tasks

1. Build a **WeatherCard**: city name (titleLarge), temperature (displaySmall), and a row with an icon and "Sunny".

2. Make a row of three IconButtons (Edit, Share, Delete), each with a proper contentDescription.

3. Build a **NotificationItem** card with a title, a two-line message (maxLines = 2, ellipsis) and a time label.

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 3

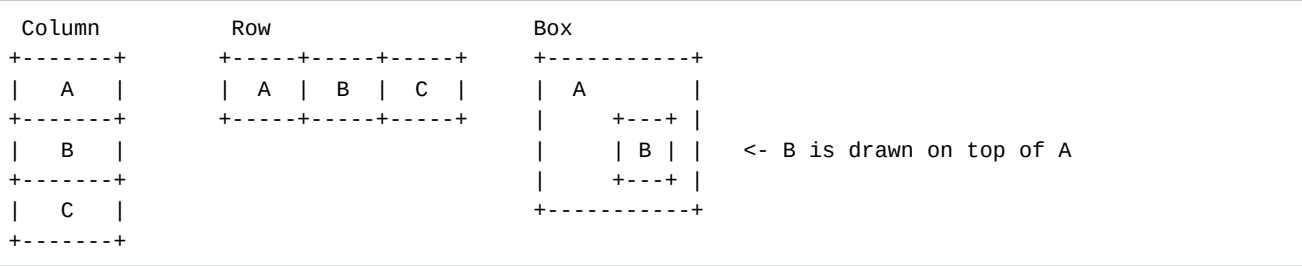
Layouts: Column, Row and Box

Goal: Arrange elements vertically, horizontally and on top of each other, and build a real list item.

1. The three basic layouts

Layout	Places children...	Like (XML)
Column	one **below** another (vertical)	LinearLayout vertical
Row	one **next to** another (horizontal)	LinearLayout horizontal
Box	**on top of** each other (stacked)	FrameLayout

Sketch: Column, Row and Box



2. Arrangement and alignment

There are two directions to control in every Column and Row:

	Main direction (arrangement)	Cross direction (alignment)
Column	verticalArrangement: Top, Center, Bottom, SpaceBetween, SpaceEvenly, spacedBy(8.dp)	horizontalAlignment: Start, CenterHorizontally, End
Row	horizontalArrangement: Start, Center, End, SpaceBetween, SpaceEvenly, spacedBy(8.dp)	verticalAlignment: Top, CenterVertically, Bottom

Arrangement examples

```
@Composable
fun ArrangementExamples() {
    Column(
        modifier = Modifier.fillMaxWidth().padding(16.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.Start) {
            Text("A"); Text("B"); Text("C")
        }
        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.Center) {
            Text("A"); Text("B"); Text("C")
        }
        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceBetween) {
            Text("A"); Text("B"); Text("C")
        }
        Row(modifier = Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.SpaceEvenly) {
            Text("A"); Text("B"); Text("C")
        }
    }
}
```

Sketch: Result

```
Start         |A B C                         |
Center        |            A B C              |
SpaceBetween  |A             B              C|
SpaceEvenly   |    A         B         C     |
```

Tip: spacedBy(8.dp) is the cleanest way to put equal gaps between items. Use it instead of a Spacer between every item.

3. weight: share the free space

Modifier.weight(1f) (inside a Row or Column) tells a child to take the remaining space. If two children have weight(1f), they share it equally.

Equal buttons and push-to-end

```
@Composable
fun WeightExamples() {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(16.dp)) {
        // Two buttons with exactly equal width
        Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
            OutlinedButton(onClick = { }, modifier = Modifier.weight(1f)) { Text("Cancel") }
            Button(onClick = { }, modifier = Modifier.weight(1f)) { Text("Save") }
        }
        // Text takes all free space, so the icon is pushed to the end
        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Notifications", modifier = Modifier.weight(1f))
            Icon(Icons.Default.Notifications, contentDescription = null)
        }
    }
}
```

4. Real example: a contact list item

This is a layout you'll build again and again at work: an avatar, two lines of text and an icon at the end.

Sketch: The design

```
+-------------------------------------------------+
| ( A )   Anjana                            [*]   |
|         98470 11111                             |
+-------------------------------------------------+
  Box     Column with weight(1f)              Icon
  (circle)
```

ContactRow

```
@Composable
fun ContactRow(name: String, phone: String, isFavorite: Boolean, modifier: Modifier = Modifier) {
    Row(
        modifier = modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        // 1. Avatar: a circle with the first letter
        Box(
            modifier = Modifier
                .size(48.dp)
                .clip(CircleShape)
                .background(MaterialTheme.colorScheme.primaryContainer),
            contentAlignment = Alignment.Center
        ) {
```

```
            Text(
                text = name.first().toString(),
                style = MaterialTheme.typography.titleMedium,
                color = MaterialTheme.colorScheme.onPrimaryContainer
            )
        }
        Spacer(Modifier.width(16.dp))
        // 2. Name and phone take all the remaining width
        Column(modifier = Modifier.weight(1f)) {
            Text(text = name, style = MaterialTheme.typography.bodyLarge)
            Text(
                text = phone,
                style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant
            )
        }
        // 3. Favourite icon at the end
        Icon(
            imageVector = if (isFavorite) Icons.Filled.Star else Icons.Outlined.Star,
            contentDescription = if (isFavorite) "Favourite" else "Not favourite",
            tint = if (isFavorite) MaterialTheme.colorScheme.primary
                   else MaterialTheme.colorScheme.outline
        )
    }
}

@Preview(showBackground = true)
@Composable
fun ContactRowPreview() {
    Column {
        ContactRow(name = "Anjana", phone = "98470 11111", isFavorite = true)
        ContactRow(name = "Rahul", phone = "90000 22222", isFavorite = false)
    }
}
```

Tip: How to build any design: draw boxes around the design on paper. Things stacked vertically = Column, side by side = Row, on top of each other = Box. Then build from the outside in.

5. Box: stacking and badges

Notification icon with a badge

```
@Composable
fun NotificationBell(count: Int) {
    Box(modifier = Modifier.padding(16.dp)) {
        Icon(
            imageVector = Icons.Default.Notifications,
            contentDescription = "Notifications",
            modifier = Modifier.size(32.dp)
        )
        if (count > 0) {
            Box(
                modifier = Modifier
                    .align(Alignment.TopEnd)
                    .size(16.dp)
                    .clip(CircleShape)
                    .background(MaterialTheme.colorScheme.error),
                contentAlignment = Alignment.Center
            ) {
                Text(
                    text = count.toString(),
                    color = MaterialTheme.colorScheme.onError,
                    style = MaterialTheme.typography.labelSmall
                )
            }
        }
    }
}
```

Notice **if (count > 0)**: in Compose you show or hide something with a normal **if**. There's no "visibility" property. (Material 3 also has a ready-made **BadgedBox** for this exact case.)

Practice Tasks

1. Build a **ProductRow**: a 64.dp square placeholder Box on the left, then name and price in a Column with weight(1f), and an "Add" button at the end.
2. Build a row of 3 equal-width stat boxes (Posts 120, Followers 2.1k, Following 180) using weight.
3. Build a profile header: a circle avatar Box with a small green "online" dot in its bottom-right corner (Box + align).

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 4

Modifiers

Goal: Control size, spacing, background, shape, borders and clicks, and understand why modifier order matters.

1. What is a Modifier?

A **Modifier** decorates a composable: its size, padding, background, shape, click behaviour and more. You chain modifiers with dots. They are applied **in order, from left to right** (top to bottom).

Modifier	What it does
padding(16.dp)	Space inside the edge. padding(horizontal = 16.dp, vertical = 8.dp) for different sides
size(48.dp) / width() / height()	Exact size
fillMaxWidth() / fillMaxSize()	Take all available width / width and height
fillMaxWidth(0.5f)	Take half the available width
weight(1f)	Share the free space (inside Row/Column only)
background(color, shape)	Background colour (optionally with a shape)
clip(shape)	Cut the content to a shape: CircleShape, RoundedCornerShape(12.dp)
border(1.dp, color, shape)	Draw a border
clickable { }	Make it tappable (with a ripple effect)
verticalScroll(rememberScrollState())	Make a Column scrollable

2. Order matters!

Same modifiers, different order

```
@Composable
fun ModifierOrder() {
    Row(modifier = Modifier.padding(16.dp), horizontalArrangement = Arrangement.spacedBy(24.dp)) {
        Text(
            text = "A",
            modifier = Modifier
                .background(MaterialTheme.colorScheme.primaryContainer)
                .padding(16.dp)
        )
        Text(
            text = "B",
            modifier = Modifier
                .padding(16.dp)
                .background(MaterialTheme.colorScheme.primaryContainer)
        )
    }
}
```

Sketch: Result

```
A: background, then padding        B: padding, then background
+-----------+                        (empty 16dp)
|           |                          +---+
|     A     |   <- colour includes     | B |   <- colour only behind the text
|           |      the padding         +---+
+-----------+
```

Think of it as: each modifier wraps everything that comes **after** it. For a tappable card with a ripple that covers the whole card, the order is usually: **size** → **clip** → **background** → **clickable** → **padding**.

3. Shapes, borders and clicks

A tappable chip and a bordered box

```
@Composable
fun ShapeExamples(onChipClick: () -> Unit) {
    Column(modifier = Modifier.padding(16.dp), verticalArrangement = Arrangement.spacedBy(16.dp)) {
        Text(
            text = "Kotlin",
            style = MaterialTheme.typography.labelLarge,
            modifier = Modifier
                .clip(RoundedCornerShape(50))                          // pill shape
                .background(MaterialTheme.colorScheme.secondaryContainer)
                .clickable { onChipClick() }
                .padding(horizontal = 16.dp, vertical = 8.dp)
        )
        Text(
            text = "Bordered box",
            modifier = Modifier
                .fillMaxWidth()
                .border(1.dp, MaterialTheme.colorScheme.outline, RoundedCornerShape(12.dp))
                .padding(16.dp)
        )
    }
}
```

4. dp and sp

Unit	Use for	Why
dp	Sizes, padding, spacing	Density-independent: looks the same physical size on every phone
sp	Text size only	Like dp, but also follows the user's font-size setting (accessibility)

Tip: Common spacing values in most design systems: **4, 8, 12, 16, 24, 32 dp**. Screen side padding is usually **16.dp**. Minimum touch target is **48.dp**.

5. The modifier parameter rule (very important in teams)

Every reusable composable should accept a **modifier** parameter, use **Modifier** as its default value, and apply it to its **outermost** element. This lets the caller decide position and size.

A reusable button following the rule

```
@Composable
fun PrimaryActionButton(
    text: String,
    onClick: () -> Unit,
    modifier: Modifier = Modifier,      // 1. first optional parameter
    enabled: Boolean = true
) {
    Button(
        onClick = onClick,
        enabled = enabled,
        modifier = modifier              // 2. applied to the root element
            .fillMaxWidth()
            .height(52.dp),
        shape = RoundedCornerShape(12.dp)
    ) {
        Text(text)
    }
}

@Composable
fun PrimaryActionButtonUsage() {
    // 3. The caller adds its own spacing
    PrimaryActionButton(text = "Continue", onClick = { }, modifier = Modifier.padding(16.dp))
}
```

Rule	Why
Parameter name is exactly **modifier**	Team convention and Android's official API guidelines
Default value is **Modifier**	Callers don't have to pass one
Required parameters first, then modifier, then other optional ones	Consistent order across the codebase
Apply it once, to the root	Applying it to inner elements gives surprising results

Practice Tasks

1. Recreate examples A and B from section 2 and look at them in the preview. Then try clip before and after background.

2. Build a tappable **CategoryChip(label, selected, onClick, modifier)** that uses primaryContainer when selected and surfaceVariant when not.

3. Make a long Column of 30 Text items scrollable with verticalScroll(rememberScrollState()).

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 5

Screen Structure with Scaffold

Goal: Build complete screens with a top app bar, floating action button, bottom navigation and correct padding.

1. What is Scaffold?

Scaffold is the frame of a Material screen. You give it the pieces, and it puts each piece in the right place:

Sketch: Scaffold slots

```
+--------------------------------+
|  topBar          (TopAppBar)   |
+--------------------------------+
|                                |
|  content  (your screen)        |
|                                |
|                        [ + ]   |  <- floatingActionButton
+--------------------------------+
|  bottomBar     (NavigationBar) |
+--------------------------------+
   snackbarHost: messages appear above the bottom bar
```

A complete home screen

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen(onAddTask: () -> Unit, onSearch: () -> Unit) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("My Tasks") },
                actions = {
                    IconButton(onClick = onSearch) {
                        Icon(Icons.Default.Search, contentDescription = "Search")
                    }
                    IconButton(onClick = { }) {
                        Icon(Icons.Default.MoreVert, contentDescription = "More options")
                    }
                }
            )
        },
        floatingActionButton = {
            FloatingActionButton(onClick = onAddTask) {
                Icon(Icons.Default.Add, contentDescription = "Add task")
            }
        },
        bottomBar = {
            NavigationBar {
                NavigationBarItem(
                    selected = true,
                    onClick = { },
                    icon = { Icon(Icons.Default.Home, contentDescription = null) },
                    label = { Text("Home") }
                )
                NavigationBarItem(
                    selected = false,
                    onClick = { },
                    icon = { Icon(Icons.Default.DateRange, contentDescription = null) },
                    label = { Text("Calendar") }
```

```
                )
                NavigationBarItem(
                    selected = false,
                    onClick = { },
                    icon = { Icon(Icons.Default.Person, contentDescription = null) },
                    label = { Text("Profile") }
                )
            }
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .padding(innerPadding)        // REQUIRED: keeps content clear of the bars
                .padding(16.dp)
        ) {
            Text("Good morning, Anjana", style = MaterialTheme.typography.headlineSmall)
            Text("You have 3 tasks today", style = MaterialTheme.typography.bodyMedium)
        }
    }
}
```

Careful: Always apply **innerPadding** to your content. If you forget it, your content is hidden **behind** the top bar and the bottom bar. This is one of the most common beginner bugs. Android Studio shows a warning if you don't use it.

@OptIn(ExperimentalMaterial3Api::class) is needed because TopAppBar is still marked "experimental" in Material 3. It's safe to use, and every team uses it.

2. Detail screen with a back button

TopAppBar with navigation icon

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun TaskDetailScreen(title: String, onBack: () -> Unit, onDelete: () -> Unit) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text(title) },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                    }
                },
                actions = {
                    IconButton(onClick = onDelete) {
                        Icon(Icons.Default.Delete, contentDescription = "Delete task")
                    }
                }
            )
        }
    ) { innerPadding ->
        Text(
            text = "Task details go here",
            modifier = Modifier.padding(innerPadding).padding(16.dp)
        )
    }
}
```

AutoMirrored icons flip automatically for right-to-left languages such as Arabic and Urdu. Use them for arrows.

Top bar	Looks like
TopAppBar	Title on the left (most common)
CenterAlignedTopAppBar	Title in the centre (main screens in many apps)
MediumTopAppBar / LargeTopAppBar	Big title that shrinks when you scroll

3. A scrollable screen body

Scrollable content inside Scaffold

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun AboutScreen(onBack: () -> Unit) {
    Scaffold(
        topBar = {
            TopAppBar(
                title = { Text("About") },
                navigationIcon = {
                    IconButton(onClick = onBack) {
                        Icon(Icons.AutoMirrored.Filled.ArrowBack, contentDescription = "Back")
                    }
                }
            )
        }
    ) { innerPadding ->
        Column(
            modifier = Modifier
                .padding(innerPadding)
                .verticalScroll(rememberScrollState())
                .padding(16.dp),
            verticalArrangement = Arrangement.spacedBy(12.dp)
        ) {
            repeat(20) { index ->
                Text("Paragraph ${index + 1}: some long text about the app...")
            }
        }
    }
}
```

Careful: Use **verticalScroll** only for a small, fixed amount of content. For long or dynamic lists, use **LazyColumn** (Chapter 6). **Never put a LazyColumn inside a verticalScroll Column**: the app crashes.

Practice Tasks

1. Build a **ProfileScreen** with a CenterAlignedTopAppBar "Profile", an Edit action icon, and a scrollable body with the ProfileCard from Chapter 1.
2. Add a fourth item "Settings" to the bottom NavigationBar of HomeScreen.
3. Find a screen with a top bar in your team's app. Is it the standard TopAppBar or a custom shared component?

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 6

Lists

Goal: Show long, scrollable lists efficiently with LazyColumn, LazyRow and LazyVerticalGrid.

1. Why "Lazy"?

A list might have 10,000 items. **LazyColumn** only creates the items that are **visible on screen**, and reuses them as you scroll. This keeps the app fast. It's the Compose version of RecyclerView.

2. Data first: a data class

Real lists show real data. Start by describing one item with a **data class** (you'll learn data classes in Week 3):

The data model and sample data

```
data class Contact(
    val id: Int,
    val name: String,
    val phone: String,
    val isFavorite: Boolean = false
)

val sampleContacts = listOf(
    Contact(1, "Anjana", "98470 11111", isFavorite = true),
    Contact(2, "Rahul", "90000 22222"),
    Contact(3, "Meera", "94470 33333", isFavorite = true),
    Contact(4, "John", "98950 44444"),
    Contact(5, "Fathima", "97450 55555"),
    Contact(6, "Arun", "96330 66666")
)
```

3. A LazyColumn

Contact list (uses Contact and sampleContacts from above)

```
@Composable
fun ContactList(
    contacts: List<Contact>,
    onContactClick: (Contact) -> Unit,
    modifier: Modifier = Modifier
) {
    LazyColumn(modifier = modifier) {
        items(items = contacts, key = { it.id }) { contact ->
            ContactItem(contact = contact, onClick = { onContactClick(contact) })
            HorizontalDivider()
        }
    }
}

@Composable
fun ContactItem(contact: Contact, onClick: () -> Unit) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(horizontal = 16.dp, vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Column(modifier = Modifier.weight(1f)) {
            Text(contact.name, style = MaterialTheme.typography.bodyLarge)
            Text(contact.phone, style = MaterialTheme.typography.bodyMedium)
        }
        if (contact.isFavorite) {
            Icon(Icons.Default.Star, contentDescription = "Favourite")
        }
    }
}

@Preview(showBackground = true)
@Composable
fun ContactListPreview() {
    ContactList(contacts = sampleContacts, onContactClick = { })
}
```

Code	Meaning
items(items = contacts) { contact -> ... }	Creates one row per item. Inside, **contact** is the current item
key = { it.id }	A unique, stable ID for each item. Keeps scroll position and animations correct when the list changes
onContactClick: (Contact) -> Unit	The list doesn't decide what happens on click. It tells its caller (state hoisting, Chapter 7)

Careful: Keys must be **unique**. Two items with the same key make the app **crash**. Use the item's database or API ID, never its position.

4. Headers, spacing and padding

Sections with headers

```
@Composable
fun SectionedContacts(contacts: List<Contact>) {
    val favorites = contacts.filter { it.isFavorite }

    LazyColumn(
        contentPadding = PaddingValues(16.dp),            // padding around the whole list
        verticalArrangement = Arrangement.spacedBy(8.dp)  // gap between items
    ) {
        item {
            Text("Favourites", style = MaterialTheme.typography.titleMedium)
        }
        items(favorites, key = { "fav-${it.id}" }) { contact ->
            Card(modifier = Modifier.fillMaxWidth()) {
                Text(contact.name, modifier = Modifier.padding(16.dp))
            }
        }
        item {
            Text("All contacts", style = MaterialTheme.typography.titleMedium)
        }
        items(contacts, key = { "all-${it.id}" }) { contact ->
            Card(modifier = Modifier.fillMaxWidth()) {
                Text(contact.name, modifier = Modifier.padding(16.dp))
            }
        }
    }
}
```

Notice the keys **"fav-1"** and **"all-1"**: the same contact appears in both sections, so the keys need a prefix to stay unique. Use **item { }** for one single element (like a header) and **items(list) { }** for many.

Tip: Use **contentPadding** instead of **Modifier.padding** on a LazyColumn. With contentPadding, items scroll smoothly to the edge of the screen instead of being cut off at the padding.

5. Horizontal lists: LazyRow

Category chips

```
@Composable
fun CategoryRow(
    categories: List<String>,
    selected: String,
    onSelect: (String) -> Unit
) {
    LazyRow(
        contentPadding = PaddingValues(horizontal = 16.dp),
        horizontalArrangement = Arrangement.spacedBy(8.dp)
    ) {
        items(categories, key = { it }) { category ->
            FilterChip(
                selected = category == selected,
                onClick = { onSelect(category) },
                label = { Text(category) }
            )
        }
    }
}
```

6. Grids: LazyVerticalGrid

A two-column product grid

```
data class Product(val id: Int, val name: String, val price: Int)

@Composable
fun ProductGrid(products: List<Product>, onProductClick: (Product) -> Unit) {
    LazyVerticalGrid(
        columns = GridCells.Fixed(2),                // or GridCells.Adaptive(160.dp)
        contentPadding = PaddingValues(16.dp),
        horizontalArrangement = Arrangement.spacedBy(12.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        items(products, key = { it.id }) { product ->
            Card(onClick = { onProductClick(product) }) {
                Column(modifier = Modifier.padding(12.dp)) {
                    Box(
                        modifier = Modifier
                            .fillMaxWidth()
                            .aspectRatio(1f)
                            .background(MaterialTheme.colorScheme.surfaceVariant)
                    )
                    Spacer(Modifier.height(8.dp))
                    Text(product.name, style = MaterialTheme.typography.titleSmall)
                    Text("Rs ${product.price}", style = MaterialTheme.typography.bodyMedium)
                }
            }
        }
    }
}
```

GridCells.Adaptive(160.dp) fits as many columns as possible, each at least 160dp wide. It looks good on both phones and tablets.

Practice Tasks

1. Create a data class **Message(id, sender, text, time)** and a LazyColumn that shows 10 sample messages, with dividers between them.

2. Add an "empty state" to ContactList: when the list is empty, show a centred Text "No contacts yet" instead of the LazyColumn.

3. Build a LazyRow of 5 "story" circles (64.dp circles with a name under each one).

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 7

State and User Input

Goal: Make screens interactive with state, handle text input and build a real login form using state hoisting.

1. What is state?

State is any value that can change and that the UI shows: text in a field, whether a checkbox is ticked, a counter. When state changes, Compose redraws (recomposes) the composables that read it.

remember and mutableStateOf

```
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }

    Column(modifier = Modifier.padding(16.dp)) {
        Text("You tapped $count times")
        Button(onClick = { count++ }) {
            Text("Tap me")
        }
    }
}
```

Part	Meaning
mutableStateOf(0)	A value that Compose **watches**. When it changes, the UI updates
remember { }	Keep the value when the function runs again. Without it, count resets to 0 on every recomposition
by	Lets you write **count** instead of **count.value**

BUG: the counter never changes

```
@Composable
fun BrokenCounter() {
    var count = 0                       // a plain variable: Compose doesn't watch it
    Button(onClick = { count++ }) {     // count changes, but nothing redraws
        Text("Tapped $count times")
    }
}
```

2. remember vs rememberSaveable

remember forgets its value when the Activity is recreated, for example when the user **rotates the phone**. **rememberSaveable** survives rotation. Use rememberSaveable for things the user typed or chose.

3. Text input

An email field

```
@Composable
fun EmailInput() {
    var email by rememberSaveable { mutableStateOf("") }

    OutlinedTextField(
        value = email,                          // what to show
        onValueChange = { email = it },         // what to do when the user types
        label = { Text("Email") },
        placeholder = { Text("name@example.com") },
        leadingIcon = { Icon(Icons.Default.Email, contentDescription = null) },
        singleLine = true,
        keyboardOptions = KeyboardOptions(
            keyboardType = KeyboardType.Email,  // shows the @ key
            imeAction = ImeAction.Next          // keyboard button says "Next"
        ),
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    )
}
```

A TextField **does not store text by itself**. You give it the current **value**, and it tells you when the user types (**onValueChange**). You save the new text in your state, Compose redraws, and the field shows the new value. This loop is the heart of Compose.

Sketch: The input loop

```
user types "a" --> onValueChange("a") --> email = "a" --> recomposition --> field shows "a"
```

4. Checkbox, Switch and RadioButton

Choice controls

```
@Composable
fun ChoiceExamples() {
    var acceptTerms by rememberSaveable { mutableStateOf(false) }
    var notifications by rememberSaveable { mutableStateOf(true) }
    var size by rememberSaveable { mutableStateOf("Medium") }

    Column(modifier = Modifier.padding(16.dp)) {
        Row(verticalAlignment = Alignment.CenterVertically) {
            Checkbox(checked = acceptTerms, onCheckedChange = { acceptTerms = it })
            Text("I accept the terms")
        }
        Row(verticalAlignment = Alignment.CenterVertically) {
            Text("Notifications", modifier = Modifier.weight(1f))
            Switch(checked = notifications, onCheckedChange = { notifications = it })
        }
        listOf("Small", "Medium", "Large").forEach { option ->
            Row(verticalAlignment = Alignment.CenterVertically) {
                RadioButton(selected = size == option, onClick = { size = option })
                Text(option)
            }
        }
    }
}
```

5. State hoisting: the most important pattern

State hoisting means moving state **up** to the caller. The child composable becomes **stateless**: it receives the **value** and an **event callback**. Rule: **state goes down, events go up**.

Sketch: State down, events up

```
            ParentScreen  (owns the state: name)
              |        ^
   value: name|        | onNameChange(newText)
              v        |
            NameField  (stateless: just shows and reports)
```

Stateless field, stateful parent

```
// Stateless: easy to reuse, preview and test
@Composable
fun NameField(name: String, onNameChange: (String) -> Unit, modifier: Modifier = Modifier) {
    OutlinedTextField(
        value = name,
        onValueChange = onNameChange,
        label = { Text("Name") },
        singleLine = true,
        modifier = modifier.fillMaxWidth()
    )
}

// Stateful: owns the state and passes it down
@Composable
fun GreetingForm() {
    var name by rememberSaveable { mutableStateOf("") }

    Column(modifier = Modifier.padding(16.dp)) {
        NameField(name = name, onNameChange = { name = it })
        Spacer(Modifier.height(8.dp))
        Text(if (name.isBlank()) "Type your name" else "Hello, $name!")
    }
}
```

Tip: Why teams love this pattern: stateless composables can be **previewed** with any data, **reused** on other screens, and **tested** easily. Most composables in a professional codebase are stateless.

6. Real example: a login form

LoginForm with validation and password toggle

```
@Composable
fun LoginForm(onLogin: (email: String, password: String) -> Unit) {
    var email by rememberSaveable { mutableStateOf("") }
    var password by rememberSaveable { mutableStateOf("") }
    var passwordVisible by rememberSaveable { mutableStateOf(false) }
    var showErrors by rememberSaveable { mutableStateOf(false) }

    val emailError = if (showErrors && !email.contains("@")) "Enter a valid email" else null
    val passwordError = if (showErrors && password.length < 6) "At least 6 characters" else null

    Column(
        modifier = Modifier.fillMaxWidth().padding(24.dp),
        verticalArrangement = Arrangement.spacedBy(12.dp)
    ) {
        Text("Welcome back", style = MaterialTheme.typography.headlineMedium)

        OutlinedTextField(
```

```
            value = email,
            onValueChange = { email = it },
            label = { Text("Email") },
            isError = emailError != null,
            supportingText = { if (emailError != null) Text(emailError) },
            singleLine = true,
            keyboardOptions = KeyboardOptions(
                keyboardType = KeyboardType.Email,
                imeAction = ImeAction.Next
            ),
            modifier = Modifier.fillMaxWidth()
        )

        OutlinedTextField(
            value = password,
            onValueChange = { password = it },
            label = { Text("Password") },
            isError = passwordError != null,
            supportingText = { if (passwordError != null) Text(passwordError) },
            singleLine = true,
            visualTransformation = if (passwordVisible) VisualTransformation.None
                                   else PasswordVisualTransformation(),
            trailingIcon = {
                IconButton(onClick = { passwordVisible = !passwordVisible }) {
                    Icon(
                        imageVector = if (passwordVisible) Icons.Default.VisibilityOff
                                      else Icons.Default.Visibility,
                        contentDescription = if (passwordVisible) "Hide password"
                                             else "Show password"
                    )
                }
            },
            keyboardOptions = KeyboardOptions(
                keyboardType = KeyboardType.Password,
                imeAction = ImeAction.Done
            ),
            modifier = Modifier.fillMaxWidth()
        )

        Button(
            onClick = {
                showErrors = true
                if (email.contains("@") && password.length >= 6) onLogin(email, password)
            },
            modifier = Modifier.fillMaxWidth().height(52.dp)
        ) {
            Text("Log in")
        }
        TextButton(onClick = { }, modifier = Modifier.align(Alignment.CenterHorizontally)) {
            Text("Forgot password?")
        }
    }
}
```

Sketch: What it looks like

```
Welcome back

+-------------------------------+
| Email                         |
+-------------------------------+
+-------------------------------+
| Password                (eye) |
+-------------------------------+
[           Log in              ]
        Forgot password?
```

Icons.Default.Visibility and **VisibilityOff** come from the **material-icons-extended** library. Ask your team if it's already added (it usually is).

In a real app, the login logic (calling the server, showing a loading spinner, handling errors) belongs in a **ViewModel**, not in the composable. That's the next chapter.

Practice Tasks

1. Add a "Confirm password" field to LoginForm that shows an error when it doesn't match the password.

2. Build a stateless **QuantitySelector(quantity, onQuantityChange)** with – and + buttons, and a stateful parent that uses it.

3. Build a sign-up form: name, email, phone (KeyboardType.Phone), a terms checkbox, and a button that's only enabled when everything is filled in.

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 8

ViewModel and UI State: The Team Pattern

Goal: Structure a screen the way professional teams do: UiState + ViewModel + a stateless screen.

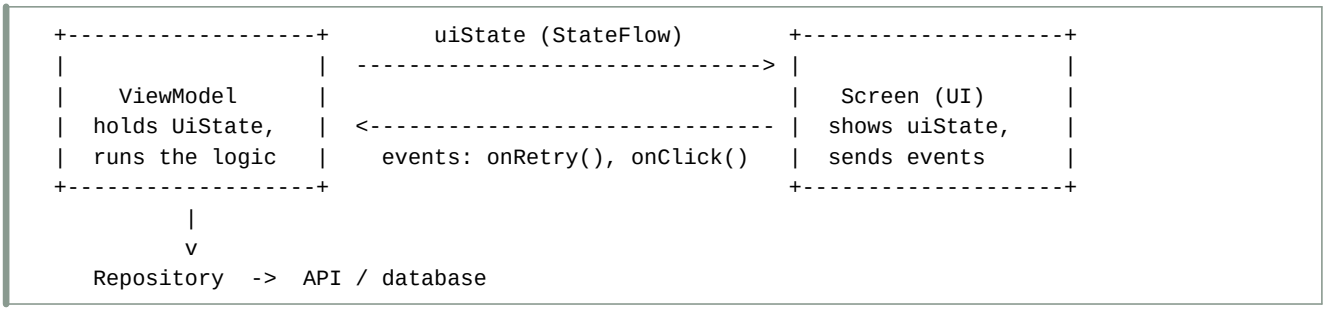
1. Why not keep everything in the composable?

In Chapter 7, state lived inside composables. That's fine for small UI state (is the password visible?). But real screens load data from a server, survive rotation, and contain business logic. That work belongs in a **ViewModel**.

Problem with logic in composables	How a ViewModel solves it
Rotation destroys the Activity, and data is lost	A ViewModel **survives** rotation
A network call inside a composable runs again on every recomposition	The ViewModel loads data once and keeps it
UI and logic are mixed and hard to test	Logic in the ViewModel can be unit-tested without a phone
Every developer does it differently	One clear pattern for the whole team

2. The architecture in one picture

Sketch: Unidirectional data flow (UDF)



This is called **unidirectional data flow**: state flows **down** from the ViewModel to the UI, and events flow **up** from the UI to the ViewModel. It's the same "state down, events up" rule from Chapter 7, at screen level.

3. Step 1: describe the UI state

First write down **everything the screen can show**. A screen that loads data usually has three states: loading, error and content. A **sealed interface** lists exactly these possibilities:

TasksUiState.kt

```
data class Task(val id: Int, val title: String, val isDone: Boolean)

sealed interface TasksUiState {
    data object Loading : TasksUiState
    data class Success(val tasks: List<Task>) : TasksUiState
    data class Error(val message: String) : TasksUiState
}
```

sealed means these are the **only** possible states, so when you use **when**, Kotlin checks that you've handled all of them. You can't forget the error state!

Tip: Some screens use a single **data class** instead: `data class LoginUiState(val email: String = "", val isLoading: Boolean = false, val error: String? = null)`. Use a data class for forms, and a sealed interface when the screen shows completely different things in each state. Follow what your team already does.

4. Step 2: the repository (where data comes from)

TaskRepository.kt (fake data for learning)

```
class TaskRepository {
    suspend fun getTasks(): List<Task> {
        delay(1000)  // pretend we're calling a server
        return listOf(
            Task(1, "Finish Week 2 lessons", isDone = true),
            Task(2, "Build the login screen", isDone = false),
            Task(3, "Review teammate's pull request", isDone = false)
        )
    }
}
```

In your team's app, repositories call the real API (often with **Retrofit**) or the database (**Room**). Screens never call the API directly.

5. Step 3: the ViewModel

TasksViewModel.kt

```
class TasksViewModel(
    private val repository: TaskRepository = TaskRepository()
) : ViewModel() {

    private val _uiState = MutableStateFlow<TasksUiState>(TasksUiState.Loading)
    val uiState: StateFlow<TasksUiState> = _uiState.asStateFlow()

    init {
        loadTasks()
    }

    fun loadTasks() {
        viewModelScope.launch {
            _uiState.value = TasksUiState.Loading
            try {
                val tasks = repository.getTasks()
                _uiState.value = TasksUiState.Success(tasks)
            } catch (e: Exception) {
                _uiState.value = TasksUiState.Error("Couldn't load tasks. Check your connection.")
            }
        }
    }

    fun onTaskCheckedChange(taskId: Int, isDone: Boolean) {
        _uiState.update { state ->
            if (state is TasksUiState.Success) {
                state.copy(tasks = state.tasks.map { task ->
                    if (task.id == taskId) task.copy(isDone = isDone) else task
                })
            } else {
                state
            }
        }
    }
}
```

Code	Meaning
_uiState (private, Mutable)	Only the ViewModel can change the state
uiState (public, read-only StateFlow)	The UI can read and observe it, but not change it
init { loadTasks() }	Start loading as soon as the ViewModel is created
viewModelScope.launch { }	Run background work (coroutines). It's cancelled automatically when the screen is closed
_uiState.update { }	Safely change the state based on its current value
state.copy(...)	Create a **new** state with changes. Never modify the old one

6. Step 4: the screen (Route + Screen)

Most teams split each screen into two composables:

- **XxxRoute** (stateful): gets the ViewModel, collects the state, passes it down. Small and boring.
- **XxxScreen** (stateless): receives **uiState** and **event lambdas**. All the UI lives here. Easy to preview.

TasksScreen.kt

```
// Stateful wrapper: connects the ViewModel to the screen
@Composable
fun TasksRoute(viewModel: TasksViewModel = viewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()

    TasksScreen(
        uiState = uiState,
        onRetry = viewModel::loadTasks,
        onTaskCheckedChange = viewModel::onTaskCheckedChange
    )
}

// Stateless screen: only shows the state and reports events
@Composable
fun TasksScreen(
    uiState: TasksUiState,
    onRetry: () -> Unit,
    onTaskCheckedChange: (taskId: Int, isDone: Boolean) -> Unit,
    modifier: Modifier = Modifier
) {
    when (uiState) {
        is TasksUiState.Loading -> {
            Box(modifier = modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
                CircularProgressIndicator()
            }
        }
        is TasksUiState.Error -> {
            Column(
                modifier = modifier.fillMaxSize().padding(24.dp),
                verticalArrangement = Arrangement.Center,
                horizontalAlignment = Alignment.CenterHorizontally
            ) {
                Text(uiState.message, textAlign = TextAlign.Center)
                Spacer(Modifier.height(16.dp))
                Button(onClick = onRetry) { Text("Retry") }
            }
        }
        is TasksUiState.Success -> {
            LazyColumn(modifier = modifier.fillMaxSize()) {
                items(uiState.tasks, key = { it.id }) { task ->
                    TaskRow(
                        task = task,
```

```
                        onCheckedChange = { checked -> onTaskCheckedChange(task.id, checked) }
                    )
                }
            }
        }
    }
}

@Composable
fun TaskRow(task: Task, onCheckedChange: (Boolean) -> Unit) {
    Row(
        modifier = Modifier.fillMaxWidth().padding(horizontal = 8.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Checkbox(checked = task.isDone, onCheckedChange = onCheckedChange)
        Text(
            text = task.title,
            textDecoration = if (task.isDone) TextDecoration.LineThrough else null
        )
    }
}
```

Code	Meaning
viewModel()	Gets (or creates) the ViewModel for this screen. It survives rotation
collectAsStateWithLifecycle()	Turns the StateFlow into Compose state, and stops collecting when the app is in the background
viewModel::loadTasks	A function reference (Week 2, Day 4) used as the onRetry lambda
when (uiState) { is ... }	Shows different UI for each state. Smart casts give you uiState.message and uiState.tasks

7. Step 5: preview every state

Because TasksScreen is stateless, you can preview all its states with fake data, without a ViewModel or a network:

Previews for each state

```
@Preview(showBackground = true, name = "Loading")
@Composable
fun TasksScreenLoadingPreview() {
    TasksScreen(uiState = TasksUiState.Loading, onRetry = { }, onTaskCheckedChange = { _, _ -> })
}

@Preview(showBackground = true, name = "Error")
@Composable
fun TasksScreenErrorPreview() {
    TasksScreen(
        uiState = TasksUiState.Error("Couldn't load tasks. Check your connection."),
        onRetry = { },
        onTaskCheckedChange = { _, _ -> }
    )
}

@Preview(showBackground = true, name = "Content")
@Composable
fun TasksScreenContentPreview() {
    TasksScreen(
        uiState = TasksUiState.Success(
            listOf(Task(1, "Buy milk", isDone = true), Task(2, "Call Amma", isDone = false))
        ),
        onRetry = { },
        onTaskCheckedChange = { _, _ -> }
    )
}
```

Tip: Reviewers in most teams expect a **preview for every state**. It's the fastest way to check loading and error UI without breaking your internet connection.

8. With Hilt (dependency injection)

Many teams use **Hilt** to give ViewModels their dependencies automatically. If your project has **@HiltViewModel** in it, your ViewModels will look like this:

Hilt version (Android only)

```
@HiltViewModel
class TasksViewModel @Inject constructor(
    private val repository: TaskRepository      // Hilt provides this
) : ViewModel() {
    // ... same as before
}

@Composable
// hiltViewModel() instead of viewModel()
fun TasksRoute(viewModel: TasksViewModel = hiltViewModel()) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
    TasksScreen(
        uiState = uiState,
        onRetry = viewModel::loadTasks,
        onTaskCheckedChange = viewModel::onTaskCheckedChange
    )
}
```

9. Checklist for a new screen

Step	File
1. List everything the screen shows: write the UiState	XxxUiState.kt
2. Write the ViewModel: StateFlow + one function per user action	XxxViewModel.kt
3. Write the stateless XxxScreen(uiState, onEvent...) with previews	XxxScreen.kt
4. Write the small XxxRoute that connects them	XxxScreen.kt
5. Add the route to the navigation graph (Chapter 9)	AppNavHost.kt

Practice Tasks

1. Add a **deleteTask(taskId)** function to TasksViewModel, and a delete IconButton to TaskRow that calls it.

2. Add an **Empty** state to TasksUiState (no tasks) and show "Nothing to do!" for it. Notice how Kotlin forces you to handle it in when.

3. Create a **ProfileUiState** data class (name, email, isLoading) and a ProfileViewModel that fills it after delay(1000).

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 9

Navigation

Goal: Move between screens with Navigation Compose, pass arguments, and handle back and login flows.

1. The parts of navigation

Part	What it is
Route	The "address" of a screen. Modern code uses @Serializable objects and classes
NavController	Remembers the back stack and moves between screens
NavHost	The container that shows the current screen, and the list of all routes (the navigation graph)
Back stack	The history of screens. Pressing Back removes the top screen

Sketch: The back stack

```
navigate(Detail(2))          back
+-----------+              +-----------+
| Detail(2) |  <- visible  |           |
+-----------+              +-----------+
| TaskList  |              | TaskList  |  <- visible again
+-----------+              +-----------+
```

2. Define the routes

Routes.kt

```
@Serializable
object TaskListRoute                              // no arguments: an object

@Serializable
data class TaskDetailRoute(val taskId: Int)       // with arguments: a data class
```

This is **type-safe navigation** (Navigation 2.8 and newer). It needs the **Kotlin serialization** plugin. Older code uses **string routes** such as "detail/{taskId}" (see section 6).

3. Build the navigation graph

AppNavHost.kt

```
@Serializable
object TaskListRoute

@Serializable
data class TaskDetailRoute(val taskId: Int)

@Composable
fun AppNavHost(navController: NavHostController = rememberNavController()) {
    NavHost(navController = navController, startDestination = TaskListRoute) {

        composable<TaskListRoute> {
            TaskListScreen(
                onTaskClick = { taskId -> navController.navigate(TaskDetailRoute(taskId)) }
            )
        }
```

```
        composable<TaskDetailRoute> { backStackEntry ->
            val route: TaskDetailRoute = backStackEntry.toRoute()
            TaskDetailScreen(
                taskId = route.taskId,
                onBack = { navController.popBackStack() }
            )
        }
    }
}

@Composable
fun TaskListScreen(onTaskClick: (Int) -> Unit) {
    LazyColumn {
        items((1..20).toList(), key = { it }) { id ->
            Text(
                text = "Task $id",
                modifier = Modifier
                    .fillMaxWidth()
                    .clickable { onTaskClick(id) }
                    .padding(16.dp)
            )
        }
    }
}

@Composable
fun TaskDetailScreen(taskId: Int, onBack: () -> Unit) {
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Details of task $taskId", style = MaterialTheme.typography.headlineSmall)
        Button(onClick = onBack) { Text("Back") }
    }
}
```

Careful: Never pass the NavController into a screen. Screens receive **lambdas** such as onTaskClick and onBack. Only the NavHost knows about navigation. This keeps screens previewable, reusable and testable, and it's what reviewers look for.

4. Common navigation actions

Login flow and other actions

```
@Serializable object LoginRoute
@Serializable object HomeRoute
@Serializable object SettingsRoute

fun navigationExamples(navController: NavHostController) {
    // Go to a screen
    navController.navigate(SettingsRoute)

    // Go back (same as the system Back button)
    navController.popBackStack()

    // After login: open Home and REMOVE Login from the back stack,
    // so pressing Back on Home closes the app instead of returning to Login
    navController.navigate(HomeRoute) {
        popUpTo<LoginRoute> { inclusive = true }
    }

    // Don't open the same screen twice if the user taps quickly
    navController.navigate(SettingsRoute) {
        launchSingleTop = true
    }
}
```

5. Navigating after a ViewModel finishes

Example: after a successful login, the ViewModel sets **isLoggedIn = true** in the UI state. The screen reacts with **LaunchedEffect**, which runs code when a value changes:

React to state with LaunchedEffect

```
data class LoginUiState(
    val email: String = "",
    val isLoading: Boolean = false,
    val isLoggedIn: Boolean = false
)

@Composable
fun LoginScreen(uiState: LoginUiState, onLoginSuccess: () -> Unit) {
    LaunchedEffect(uiState.isLoggedIn) {
        if (uiState.isLoggedIn) {
            onLoginSuccess()            // NavHost decides where to go
        }
    }
    // ... the login form UI ...
}
```

6. Older code: string routes

String routes (you may see these in existing code)

```
NavHost(navController = navController, startDestination = "tasks") {
    composable("tasks") {
        TaskListScreen(onTaskClick = { id -> navController.navigate("tasks/$id") })
    }
    composable(
        route = "tasks/{taskId}",
        arguments = listOf(navArgument("taskId") { type = NavType.IntType })
    ) { backStackEntry ->
        val taskId = backStackEntry.arguments?.getInt("taskId") ?: 0
        TaskDetailScreen(taskId = taskId, onBack = { navController.popBackStack() })
    }
}
```

Tip: Pass only **IDs** between screens (taskId), never whole objects. The next screen's ViewModel loads the full data using the ID. This is the official recommendation.

Practice Tasks

1. Add a **SettingsRoute** to AppNavHost and a settings icon in TaskListScreen's top bar that opens it.

2. Build a 3-screen flow: Login → Home → Profile. Make sure Back on Home closes the app (popUpTo inclusive).

3. Open your team's navigation file. Are routes type-safe objects or strings? How are arguments passed?

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 10

Theme, Colours and Strings

Goal: Use the design system correctly: colour and typography tokens, string resources and dark mode.

1. The theme is your design system

The **ui/theme** folder defines the app's colours (Color.kt), fonts (Type.kt) and the theme function (Theme.kt). Screens **read** from the theme through **MaterialTheme**. They should never invent their own colours.

Theme.kt (simplified, Android only)

```
private val LightColors = lightColorScheme(
    primary = Color(0xFF1E7A4C),
    onPrimary = Color.White,
    secondary = Color(0xFF6B4FD8),
    // ...
)

private val DarkColors = darkColorScheme(
    primary = Color(0xFF4CC68A),
    onPrimary = Color(0xFF0C1610),
    // ...
)

@Composable
fun MyAppTheme(darkTheme: Boolean = isSystemInDarkTheme(), content: @Composable () -> Unit) {
    MaterialTheme(
        colorScheme = if (darkTheme) DarkColors else LightColors,
        typography = AppTypography,
        content = content
    )
}
```

2. Colour tokens

Token	Use for
primary / onPrimary	Main brand colour (buttons, active states) / text and icons on top of it
primaryContainer / onPrimaryContainer	Softer brand background (chips, avatars, highlighted cards)
secondary, tertiary (+ containers)	Accents
surface / onSurface	Screen and card backgrounds / normal text
surfaceVariant / onSurfaceVariant	Subtle backgrounds / secondary text such as subtitles
outline / outlineVariant	Borders and dividers
error / onError	Errors and destructive actions

Every colour has an **on-** partner: always put **onX** content on an **X** background. Then text stays readable in both light and dark mode.

BAD: hardcoded colours break dark mode

```
Text("Hello", color = Color(0xFF000000))       // black text: invisible in dark mode
Box(Modifier.background(Color(0xFFFFFFFF)))     // always white
```

GOOD: theme tokens work in light and dark

```
@Composable
fun ThemedInfoCard(title: String, message: String) {
    Surface(
        color = MaterialTheme.colorScheme.surfaceVariant,
        shape = RoundedCornerShape(12.dp),
        modifier = Modifier.fillMaxWidth().padding(16.dp)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(title, style = MaterialTheme.typography.titleMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
            Text(message, style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
        }
    }
}
```

Tip: Inside a Surface or Card, Compose already chooses the matching **on-** colour for text, so you often don't need to set the text colour at all.

3. Strings belong in strings.xml

Professional apps never hardcode user-visible text in Kotlin. Text lives in **res/values/strings.xml**, so it can be **translated** (Malayalam, Hindi, Arabic...) and changed without touching code.

res/values/strings.xml

```
<resources>
    <string name="app_name">My Tasks</string>
    <string name="login_title">Welcome back</string>
    <string name="login_button">Log in</string>
    <string name="greeting">Hello, %1$s!</string>
    <plurals name="tasks_count">
        <item quantity="one">%d task</item>
        <item quantity="other">%d tasks</item>
    </plurals>
</resources>
```

Using string resources in Compose (Android only)

```
Text(text = stringResource(R.string.login_title))
Text(text = stringResource(R.string.greeting, userName))              // "Hello, Anjana!"
Text(text = pluralStringResource(R.plurals.tasks_count, count, count)) // "1 task" / "5 tasks"
```

res/values-ml/strings.xml (Malayalam translation)

```
<resources>
    <string name="login_title">Swagatham</string>   <!-- typed in Malayalam script -->
</resources>
```

Android automatically picks the **values-ml** file when the phone's language is Malayalam. The code doesn't change at all.

Tip: Shortcut: put the cursor on a hardcoded string in Kotlin, press **Alt + Enter** → **Extract string resource**. Android Studio moves it to strings.xml for you.

4. Preview in dark mode

Light and dark previews (Android only)

```
@Preview(name = "Light", showBackground = true)
@Preview(name = "Dark", uiMode = Configuration.UI_MODE_NIGHT_YES, showBackground = true)
@Composable
fun ThemedInfoCardPreview() {
    MyAppTheme {
        ThemedInfoCard(title = "Tip", message = "Drink water regularly")
    }
}
```

5. Turning a Figma design into code

In Figma	In Compose
Frames stacked vertically / horizontally (auto layout)	Column / Row with spacedBy(gap)
Padding values (e.g. 16)	padding(16.dp). Figma pixels at 1x = dp
Font style names (e.g. "Title/Medium")	MaterialTheme.typography.titleMedium (or your team's tokens)
Colour styles (e.g. "Primary", "Surface Variant")	MaterialTheme.colorScheme.primary, .surfaceVariant
Corner radius 12	RoundedCornerShape(12.dp)
Components (buttons, cards)	Look for an existing composable in ui/components first

Tip: If a colour or size in the design doesn't match any token in your theme, **ask the designer** before adding it. It might be a mistake in the design, or a new token the whole team should add.

Practice Tasks

1. Find 3 hardcoded colours or strings in your practice project and replace them with theme tokens and string resources.

2. Add a **values-ml/strings.xml** to your practice project, translate 2 strings, and switch the emulator's language to Malayalam to see them.

3. Open your team's Theme.kt and Color.kt. Write down the primary, surface and error colours for light and dark mode.

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 11

Real-World Screen Recipes

Goal: Build the patterns you'll meet in almost every app: settings, search, dialogs, snackbars, empty and loading states.

1. Settings screen

Settings rows

```
@Composable
fun SettingsSwitchRow(
    title: String,
    subtitle: String,
    checked: Boolean,
    onCheckedChange: (Boolean) -> Unit
) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable { onCheckedChange(!checked) }
            .padding(horizontal = 16.dp, vertical = 12.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Column(modifier = Modifier.weight(1f)) {
            Text(title, style = MaterialTheme.typography.bodyLarge)
            Text(subtitle, style = MaterialTheme.typography.bodyMedium,
                color = MaterialTheme.colorScheme.onSurfaceVariant)
        }
        Switch(checked = checked, onCheckedChange = onCheckedChange)
    }
}

@Composable
fun SettingsNavigationRow(title: String, icon: ImageVector, onClick: () -> Unit) {
    Row(
        modifier = Modifier
            .fillMaxWidth()
            .clickable(onClick = onClick)
            .padding(16.dp),
        verticalAlignment = Alignment.CenterVertically
    ) {
        Icon(icon, contentDescription = null)
        Spacer(Modifier.width(16.dp))
        Text(title, modifier = Modifier.weight(1f), style = MaterialTheme.typography.bodyLarge)
        Icon(Icons.AutoMirrored.Filled.KeyboardArrowRight, contentDescription = null)
    }
}

@Composable
fun SettingsContent() {
    var darkMode by rememberSaveable { mutableStateOf(false) }
    var notifications by rememberSaveable { mutableStateOf(true) }

    Column {
        SettingsSwitchRow("Dark mode", "Use a dark theme", darkMode) { darkMode = it }
        SettingsSwitchRow("Notifications", "Daily reminders", notifications) { notifications = it }
        HorizontalDivider()
        SettingsNavigationRow("Account", Icons.Default.Person) { }
        SettingsNavigationRow("Privacy", Icons.Default.Lock) { }
        SettingsNavigationRow("Help", Icons.Default.Info) { }
```

```
    }
}
```

Notice that the **whole row** is clickable, not only the small switch. Users expect to tap anywhere on the row.

2. Search with filtering

Search bar + filtered list

```
@Composable
fun SearchableList(allItems: List<String>) {
    var query by rememberSaveable { mutableStateOf("") }
    val filtered = remember(query, allItems) {
        allItems.filter { it.contains(query, ignoreCase = true) }
    }

    Column {
        OutlinedTextField(
            value = query,
            onValueChange = { query = it },
            placeholder = { Text("Search") },
            leadingIcon = { Icon(Icons.Default.Search, contentDescription = null) },
            trailingIcon = {
                if (query.isNotEmpty()) {
                    IconButton(onClick = { query = "" }) {
                        Icon(Icons.Default.Clear, contentDescription = "Clear search")
                    }
                }
            },
            singleLine = true,
            modifier = Modifier.fillMaxWidth().padding(16.dp)
        )
        if (filtered.isEmpty()) {
            Text("No results for \"$query\"", modifier = Modifier.padding(16.dp))
        } else {
            LazyColumn {
                items(filtered, key = { it }) { item ->
                    Text(item, modifier = Modifier.fillMaxWidth().padding(16.dp))
                }
            }
        }
    }
}
```

remember(query, allItems) { ... } only re-runs the filter when **query** or **allItems** changes, not on every recomposition. In a real app, searching usually happens in the ViewModel.

3. Confirmation dialog

Delete confirmation

```
@Composable
fun DeleteTaskButton(taskTitle: String, onConfirmDelete: () -> Unit) {
    var showDialog by remember { mutableStateOf(false) }

    IconButton(onClick = { showDialog = true }) {
        Icon(Icons.Default.Delete, contentDescription = "Delete")
    }

    if (showDialog) {
        AlertDialog(
            onDismissRequest = { showDialog = false },     // tap outside or press Back
            title = { Text("Delete task?") },
            text = { Text("\"$taskTitle\" will be permanently deleted.") },
            confirmButton = {
                TextButton(onClick = {
                    showDialog = false
                    onConfirmDelete()
                }) { Text("Delete") }
            },
            dismissButton = {
                TextButton(onClick = { showDialog = false }) { Text("Cancel") }
            }
        )
    }
}
```

4. Snackbar messages

Show a snackbar with Undo

```
@Composable
fun SnackbarExample() {
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()

    Scaffold(snackbarHost = { SnackbarHost(snackbarHostState) }) { innerPadding ->
        Button(
            onClick = {
                scope.launch {
                    val result = snackbarHostState.showSnackbar(
                        message = "Task deleted",
                        actionLabel = "Undo"
                    )
                    if (result == SnackbarResult.ActionPerformed) {
                        // restore the task
                    }
                }
            },
            modifier = Modifier.padding(innerPadding).padding(16.dp)
        ) {
            Text("Delete task")
        }
    }
}
```

showSnackbar is a **suspend** function: it waits until the snackbar disappears. That's why it runs inside **scope.launch**, using a scope from **rememberCoroutineScope()**.

5. Reusable empty, error and loading states

Shared state composables (put these in ui/components)

```
@Composable
fun LoadingState(modifier: Modifier = Modifier) {
    Box(modifier = modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
        CircularProgressIndicator()
    }
}

@Composable
fun EmptyState(icon: ImageVector, title: String, message: String, modifier: Modifier = Modifier) {
    Column(
        modifier = modifier.fillMaxSize().padding(32.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Icon(icon, contentDescription = null, modifier = Modifier.size(64.dp),
            tint = MaterialTheme.colorScheme.outline)
        Spacer(Modifier.height(16.dp))
        Text(title, style = MaterialTheme.typography.titleMedium)
        Text(message, style = MaterialTheme.typography.bodyMedium, textAlign = TextAlign.Center,
            color = MaterialTheme.colorScheme.onSurfaceVariant)
    }
}

@Composable
fun ErrorState(message: String, onRetry: () -> Unit, modifier: Modifier = Modifier) {
    Column(
        modifier = modifier.fillMaxSize().padding(32.dp),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Icon(Icons.Default.Warning, contentDescription = null, modifier = Modifier.size(64.dp),
            tint = MaterialTheme.colorScheme.error)
        Spacer(Modifier.height(16.dp))
        Text(message, textAlign = TextAlign.Center)
        Spacer(Modifier.height(16.dp))
        Button(onClick = onRetry) { Text("Try again") }
    }
}
```

6. Button with a loading state

Loading button

```
@Composable
fun LoadingButton(
    text: String,
    isLoading: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Button(
        onClick = onClick,
        enabled = !isLoading,                       // prevent double taps
        modifier = modifier.fillMaxWidth().height(52.dp)
    ) {
        if (isLoading) {
            CircularProgressIndicator(
                modifier = Modifier.size(20.dp),
                strokeWidth = 2.dp,
                color = MaterialTheme.colorScheme.onPrimary
            )
        } else {
            Text(text)
        }
    }
}
```

Practice Tasks

1. Build a full **SettingsScreen** with a Scaffold, TopAppBar and SettingsContent, plus a "Log out" row that shows a confirmation dialog.

2. Replace the loading and error UI in TasksScreen (Chapter 8) with the shared LoadingState and ErrorState composables.

3. Use LoadingButton in LoginForm: show the spinner for 2 seconds after tapping (use LaunchedEffect or a coroutine with delay).

Stuck? Send your code to your mentor and ask for a review.

CHAPTER 12

Working in a Team

Goal: Find your way in a big codebase, follow conventions, avoid common mistakes and get your pull requests approved.

1. How to find the code for a screen

Method	How
Search for visible text	Note some text on the screen (e.g. "Forgot password?"). Press **Ctrl + Shift + F** and search for it. You'll find it in strings.xml. Then right-click the string name → **Find Usages** (Alt + F7) to find the composable
Layout Inspector	Run the app, open **Tools** → **Layout Inspector**, and click on any element. It shows the composable tree and the file name of the composable
Navigation graph	Open the NavHost file and find the route. The composable called for that route is the screen
Search by name	Press **Shift twice** and type part of the name, e.g. "ProfileScreen"

2. Workflow for a UI ticket

- **1. Understand**: read the ticket and open the Figma design. Ask questions **before** coding.
- **2. Branch**: `git checkout -b feature/profile-screen` (use your team's naming).
- **3. Reuse**: check ui/components and similar screens for existing pieces.
- **4. Build the stateless screen first**, with fake data and previews for every state.
- **5. Connect** the ViewModel and navigation.
- **6. Test on a device**: light/dark mode, rotation, a small phone, large font size, slow network.
- **7. Self-review** your diff with the checklist below, then open the pull request.

3. Common team conventions

Convention	Example
Screen composables end with Screen	ProfileScreen, OrderDetailScreen
The stateful wrapper ends with Route	ProfileRoute
ViewModel and state names match the screen	ProfileViewModel, ProfileUiState
One screen per file, grouped by feature	feature/profile/ProfileScreen.kt
Every composable that emits UI has a modifier parameter	fun Avatar(url: String, modifier: Modifier = Modifier)
Events are named onSomething	onLoginClick, onEmailChange, onBack
No hardcoded strings, colours or sizes in screens	stringResource(), MaterialTheme.colorScheme
Previews for each state	Loading, Error, Content, Empty

4. Accessibility basics

- Give icons and images a **contentDescription** (or null if they're decorative).
- Make touch targets at least **48.dp**. IconButton does this for you.
- Use theme colours so text has enough **contrast** in light and dark mode.
- Test with **large font size** (Settings → Display → Font size). Don't give text containers a fixed height.
- Turn on **TalkBack** once in a while and try using your screen without looking.

5. Performance basics

Do	Don't
Give lazy list items a stable **key**	Use the list position as the key
Load and process data in the **ViewModel**	Call APIs or sort big lists directly inside a composable
Use **remember** for expensive calculations	Recalculate on every recomposition
Pass only the data a composable needs	Pass the whole ViewModel down to child composables
Use LazyColumn for long lists	Use Column + verticalScroll for hundreds of items

6. Common mistakes and fixes

Mistake	What happens	Fix
Forgetting innerPadding from Scaffold	Content hidden under the bars	Apply Modifier.padding(innerPadding)
LazyColumn inside a verticalScroll Column	Crash: "infinite height constraints"	Use one LazyColumn with item { } for the other content
var count = 0 without remember/mutableStateOf	UI never updates	var count by remember { mutableStateOf(0) }
Changing a list in place (list.add) in state	UI doesn't update	Create a new list: tasks = tasks + newTask
Passing NavController into screens	Hard to preview, test and reuse	Pass lambdas: onBack, onItemClick
Duplicate keys in a LazyColumn	Crash	Use unique IDs, with a prefix per section
Hardcoded colour Color.Black for text	Invisible text in dark mode	Use MaterialTheme.colorScheme.onSurface
Business logic in composables	Runs again on recomposition, lost on rotation	Move it into the ViewModel

7. Pull request self-review checklist

Check	Done?
The screen matches the design (spacing, fonts, colours) in light **and** dark mode	[ ]
No hardcoded strings, colours or magic numbers; existing components are reused	[ ]
Screen is stateless with previews for every state; Route connects the ViewModel	[ ]
Rotation keeps the user's input and data	[ ]
Loading, error and empty states are handled	[ ]
Content descriptions added; touch targets at least 48dp	[ ]
No leftover Log statements, commented-out code or TODOs without a ticket	[ ]
The project builds, and lint shows no new warnings	[ ]
The PR description explains **what** and **why**, with screenshots	[ ]

8. Git commands you'll use every day

Command	What it does
git pull	Get the latest code from the team
git checkout -b feature/my-screen	Create a new branch for your task
git status	See which files you changed
git add .	Stage all your changes
git commit -m "Add profile screen UI"	Save a snapshot with a message
git push -u origin feature/my-screen	Upload your branch, then open a pull request

Tip: Android Studio has all of this built in: the **Commit** panel (Ctrl + K) and **Git** → **Push** (Ctrl + Shift + K). Small, frequent commits with clear messages make reviews much easier.

9. Getting help effectively

- Try for 20 to 30 minutes on your own first: read the error, search the codebase for similar code.
- When you ask, include: what you're trying to do, what you tried, the exact error message and a screenshot.
- Read your teammates' pull requests. It's the fastest way to learn the codebase and the team's style.
- Keep a personal notes file of things you learn (commands, patterns, where things are).

CHAPTER 13

Reading XML Layouts (Older Code)

Goal: Understand and edit screens built with XML layouts, ViewBinding and RecyclerView, and mix them with Compose.

1. When will you see XML?

Before Compose (2021), all Android screens were built with **XML layout files** in **res/layout/**. Many apps still have XML screens, often alongside newer Compose screens. You'll need to **read** and **edit** them.

2. A simple XML layout

res/layout/activity_login.xml

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:id="@+id/titleText"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/login_title"
        android:textAppearance="?attr/textAppearanceHeadlineMedium" />

    <EditText
        android:id="@+id/emailInput"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:hint="@string/email"
        android:inputType="textEmailAddress" />

    <Button
        android:id="@+id/loginButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:text="@string/login_button" />
</LinearLayout>
```

Attribute	Meaning
layout_width / layout_height	Required on every view: **match_parent** (fill the parent), **wrap_content** (as big as the content) or a size like 48dp
android:id="@+id/loginButton"	A name so Kotlin code can find the view
orientation="vertical"	LinearLayout direction (vertical = like Column, horizontal = like Row)
padding / layout_margin	Space inside the view / space outside the view
@string/login_title	Text from strings.xml
gravity / layout_gravity	Align the content inside a view / align the view inside its parent
visibility	visible, invisible (hidden but takes space) or gone (hidden, takes no space)

3. ConstraintLayout

Most XML screens use **ConstraintLayout**: each view is attached ("constrained") to the parent or to other views.

ConstraintLayout example

```
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:padding="16dp">

    <ImageView
        android:id="@+id/avatar"
        android:layout_width="48dp"
        android:layout_height="48dp"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/nameText"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="16dp"
        app:layout_constraintStart_toEndOf="@id/avatar"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintTop_toTopOf="@id/avatar" />
</androidx.constraintlayout.widget.ConstraintLayout>
```

Read **layout_constraintStart_toEndOf="@id/avatar"** as "my start edge is attached to the avatar's end edge". **layout_width="0dp"** means "fill the space between my constraints" (like weight(1f)).

4. Connecting XML to Kotlin with ViewBinding

Activity with ViewBinding (Android only)

```
class LoginActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLoginBinding   // generated from activity_login.xml

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)

        binding.loginButton.setOnClickListener {
            val email = binding.emailInput.text.toString()
            if (email.contains("@")) {
                binding.titleText.text = "Logging in..."
            } else {
                binding.emailInput.error = "Enter a valid email"
            }
        }
    }
}
```

ViewBinding generates a class from the file name (activity_login.xml → **ActivityLoginBinding**) with one property per view id (login_button or loginButton → **binding.loginButton**). Very old code uses **findViewById** instead.

5. Fragments

Fragment with ViewBinding (Android only)

```
class ProfileFragment : Fragment(R.layout.fragment_profile) {

    private var _binding: FragmentProfileBinding? = null
    private val binding get() = _binding!!

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        _binding = FragmentProfileBinding.bind(view)
        binding.nameText.text = "Anjana"
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null          // important: avoid memory leaks
    }
}
```

6. Lists in XML: RecyclerView

A RecyclerView adapter (Android only)

```
class ContactAdapter(
    private val onClick: (Contact) -> Unit
) : ListAdapter<Contact, ContactAdapter.ViewHolder>(DiffCallback) {

    class ViewHolder(val binding: ItemContactBinding) : RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val binding = ItemContactBinding.inflate(LayoutInflater.from(parent.context), parent, false)
        return ViewHolder(binding)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val contact = getItem(position)
        holder.binding.nameText.text = contact.name
        holder.binding.root.setOnClickListener { onClick(contact) }
    }

    object DiffCallback : DiffUtil.ItemCallback<Contact>() {
        override fun areItemsTheSame(old: Contact, new: Contact) = old.id == new.id
        override fun areContentsTheSame(old: Contact, new: Contact) = old == new
    }
}

// In the Fragment or Activity:
// binding.recyclerView.adapter = adapter
// adapter.submitList(contacts)
```

Compare this with LazyColumn in Chapter 6. Compose needs **far less code** for the same result, which is why teams are moving to it.

7. Mixing Compose and XML

Compose inside an XML screen, and XML views inside Compose (Android only)

```
// 1. Compose inside a Fragment (common when migrating screen by screen)
override fun onCreateView(inflater: LayoutInflater, container: ViewGroup?, state: Bundle?): View {
    return ComposeView(requireContext()).apply {
        setContent {
            MyAppTheme { ProfileScreen() }
        }
    }
}

// 2. An old custom View inside Compose
@Composable
fun LegacyChart(data: List<Float>) {
    AndroidView(
        factory = { context -> ChartView(context) },     // created once
        update = { chartView -> chartView.setData(data) } // called when data changes
    )
}
```

8. XML to Compose translation table

XML (View system)	Compose
LinearLayout (vertical / horizontal)	Column / Row
FrameLayout	Box
ConstraintLayout	Row/Column/Box (or ConstraintLayout for Compose)
RecyclerView + Adapter	LazyColumn / LazyRow / LazyVerticalGrid
ScrollView	Column + verticalScroll
TextView / EditText / Button / ImageView	Text / TextField / Button / Image
layout_weight="1"	Modifier.weight(1f)
visibility="gone"	if (visible) { ... }
setOnClickListener { }	onClick = { } or Modifier.clickable { }
@string/name, @color/name, @dimen/name	stringResource(), MaterialTheme.colorScheme, dp values or tokens

Practice Tasks

1. Find one XML layout in your team's project. Draw its structure as a tree (like Chapter 1) and write the equivalent Compose code.

2. In your practice project, create a ComposeView inside an XML layout or Fragment and show the ContactRow from Chapter 3 in it.

3. Translate the activity_login.xml example in this chapter into a Compose LoginScreen.

Stuck? Send your code to your mentor and ask for a review.

Appendix A: Common imports

Android Studio adds imports when you press **Alt + Enter**, but these are the ones you'll see most often:

Package	Contains
androidx.compose.runtime.*	@Composable, remember, mutableStateOf, rememberSaveable (runtime.saveable), LaunchedEffect, getValue/setValue (for by)
androidx.compose.foundation.layout.*	Column, Row, Box, Spacer, padding, fillMaxWidth, size, Arrangement, PaddingValues
androidx.compose.foundation.*	background, border, clickable, Image, verticalScroll, rememberScrollState
androidx.compose.foundation.lazy.*	LazyColumn, LazyRow, items
androidx.compose.foundation.shape.*	RoundedCornerShape, CircleShape
androidx.compose.material3.*	Text, Button, Card, Scaffold, TopAppBar, OutlinedTextField, MaterialTheme...
androidx.compose.material.icons.Icons + .filled.* / .outlined.*	Icons.Default.Home, Icons.Outlined.Star...
androidx.compose.ui.Modifier / Alignment	Modifier, Alignment
androidx.compose.ui.unit.dp / sp	dp and sp units
androidx.lifecycle.viewmodel.compose.viewModel	viewModel()
androidx.lifecycle.compose.collectAsStateWithLifecycle	collectAsStateWithLifecycle()
androidx.navigation.compose.*	NavHost, composable, rememberNavController

Appendix B: Dependencies (build.gradle.kts)

Your team's project already has most of these. Use the versions from your project's **libs.versions.toml**. Don't upgrade versions on your own (remember the AAR metadata error!).

Library	Needed for
androidx.compose:compose-bom + ui, material3, ui-tooling-preview	Compose itself and previews
androidx.activity:activity-compose	setContent in an Activity
androidx.lifecycle:lifecycle-viewmodel-compose	viewModel()
androidx.lifecycle:lifecycle-runtime-compose	collectAsStateWithLifecycle()
androidx.navigation:navigation-compose + Kotlin serialization plugin	Navigation with type-safe routes
androidx.compose.material:material-icons-extended	Extra icons (Visibility, CameraAlt...)
io.coil-kt.coil3:coil-compose	Loading images from URLs (AsyncImage)
com.google.dagger:hilt-android + androidx.hilt:hilt-navigation-compose	Hilt and hiltViewModel()

Appendix C: Quick reference

I want to...	Use
Show text	Text(text, style = MaterialTheme.typography.bodyLarge)
Stack vertically / horizontally / on top	Column / Row / Box
Equal gaps between children	Arrangement.spacedBy(8.dp)
Fill the remaining space	Modifier.weight(1f)
Round corners / a circle	Modifier.clip(RoundedCornerShape(12.dp)) / clip(CircleShape)
Make something tappable	Modifier.clickable { }
A long list	LazyColumn { items(list, key = { it.id }) { ... } }
Remember a value	var x by rememberSaveable { mutableStateOf(...) }
Text input	OutlinedTextField(value, onValueChange, label)
Screen frame with bars	Scaffold(topBar, bottomBar, floatingActionButton) { innerPadding -> }
Show data from a ViewModel	val state by viewModel.uiState.collectAsStateWithLifecycle()
Run code when a value changes	LaunchedEffect(key) { ... }
Go to another screen	navController.navigate(DetailRoute(id))
Show or hide something	if (condition) { ... }

Next: Work through one chapter at a time, and compare it with your team's code as you go. Send any chapter's practice tasks to your mentor for a review. Good luck, Anjana!